

Deep Learning Final Project Report

Alon Malka Amit Arad

February 2026

Abstract

Music Source Separation (MSS) is the task of isolating an individual sound source from a mixed audio signal. This project implements and compares neural network architectures for spectrogram-based vocal separation: U-Net codecs and LSTM-based models. All operated on the log-magnitude spectrograms of the data and trained via curriculum learning ($2 \rightarrow 1$ into $4 \rightarrow 3$ source separation). Our results show that U-Net models outperform the LSTM baseline across all separation quality metrics (SDR, SIR, SAR). Additionally, we experimented with adding attention mechanisms to the Codec.

1 Introduction

Vocal separation is a sub-task of Music Source Separation (MSS), which means decomposing a mixed audio signal into its separate tracks (vocals, guitars, piano, etc.) It is still being researched and is our main objective in the final project. Our preliminary work was to examine recent works and research in the field. We concluded that most approaches use U-Nets or LSTMs based models to solve this problem. The idea focuses on modeling image-to-image mappings, where the input image is the spectrogram of the song and the output is a predicted mask, to be multiplied by the input mixture in order to mask out the vocals. In our project, we decided to focus on the U-Net architecture, implementing a standard U-Net and comparing it with the LSTM model. Additionally, we tried equipping the resulting U-Net architecture with a multi-head attention block between the encoder and the decoder parts. Training was performed on a data set named "MUSDB18", a standard in the field. It contains approximately 10 hours of songs and their 4 different stems.

2 Related Work

The first work, of Derby et al. [1], motivated working with mask-based approaches for music source separation.

3 Methodology

In this section we will describe the ideas that led us to the specific architectures we implemented, the main pipeline and flow of the project. Naturally, we started with the data and the way we processed it, then we moved to the models and training.

3.1 The Data

Background In digital signal processing (DSP), analyzing the raw time-domain signal $x[n]$ is usually inefficient. This is why the Discrete Fourier Transform (DFT) is being used:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i2\pi k \frac{n}{N}}$$

While the DFT reveals the frequency composition of a signal it lacks temporal resolution, since it averages frequency across the time domain. To capture the frequency composition as a function of time, we use the STFT. By using a sliding window function $w[n]$, we calculate the Fourier transform for small segments of the signal:

$$X[m, k] = \sum_{n=0}^{N-1} x[n + m \cdot H] \cdot w[n] e^{-i2\pi k \frac{n}{N}} \in \mathbb{C}$$

The magnitude of $X[m, k]$ is called the spectrogram (an illustration can be seen in Fig. 1). It is common to omit the phase (the angle of the complex number) and use only its magnitude during analysis since the phase prediction is a more complicated task, due to its cyclic nature. The phase is kept from the original signal and is only used for reconstruction of the audio signal. By converting audio into a spectrogram we essentially reduce the task to processing 2D images. This allows us to leverage computer vision architectures taught in class, like CNNs and U-Nets for our task.

The Data Set We used a database called MUSDB18 which is a standard in the field. It has approximately 10 hours of songs (signals). The signals are stereophonic and encoded at 44.1 kHz (songs on most popular music streaming are encoded at this rate). Each song has five tracks (stems): 'vocals', 'drums', 'bass', 'other' and 'mixture' as can be seen in Fig. 2. As input, we take a linear combination of these 4 stems (the initial idea, which was later abandoned, was to assign changing weight ratios as the coefficients of this linear combination in order for the model to be more robust when facing various types of songs), and as for the output, we predict a mask of the same input size (with values ranging from 0 to 1), that is to be multiplied by the input spectrogram elementwise to isolate the vocals.

Data processing Working with audio files can be challenging due to their size. For a simple 3 minutes song sampled at 44.1kHz, one must allocate memory space proportional to $44100 \cdot 180 \approx 8M$ which is huge. To overcome this, we downsample the audio by a factor of 2, converting it to 22.05kHz. We then split the musdb18 songs into chunks of 8 seconds with 4 seconds overlap, i.e., some 16 seconds of a song will be divided into three batches, 0-8, 4-12, 8-16. Then, the STFT of a chunk is being calculated, with $n_fft = 2048$ and $hop_length = 512$, resulting in a 1025×341 spectrograms that serve as input to the model.

3.2 Models

U-Net Our main model in this project was the U-Net. As mentioned, the idea is the via signal processing tools, we convert the task to image-to-image translation, hence it was natural to use a convolutional based architecture. Essentially, the U-Net follows an Encoder-Decoder (Codec) architecture. The encoder consists of down-sample convolutional blocks reducing spatial information. The decoder does the opposite with up-sample blocks. In addition, between each layer we added skip-connections that resemble a residual block, as can be seen in Fig. 3, to allow the model to learn the identity mapping.

LSTM The baseline architecture was taken from the work of Derby et al. [1]. Using torch library, we implemented a bidirectional LSTM-based model, feeding it with the spectrograms as sequential input.

U-Net + Multihead attention block After implementing the U-Net model, we wanted to improve its performance. To do so, we decided to add multi-head attentional blocks.

3.3 Training

We experimented with curriculum learning, i.e., trained the model in two separate stages. The first was $2 \rightarrow 1$ source separation. The vocals + one other random track were merged and given as input. After 50 epochs, stage two was initialized with the task of $4 \rightarrow 1$ source separation. The model continued training with the weights it had from stage one.

4 Experiments

5 Work Report

References

- [1] B. Derby, L. Dunker, S. Galchar, S. Jarmale, and A. Setti, “Source Separation & Automatic Transcription for Music,” Submitted on 9 Dec 2024.

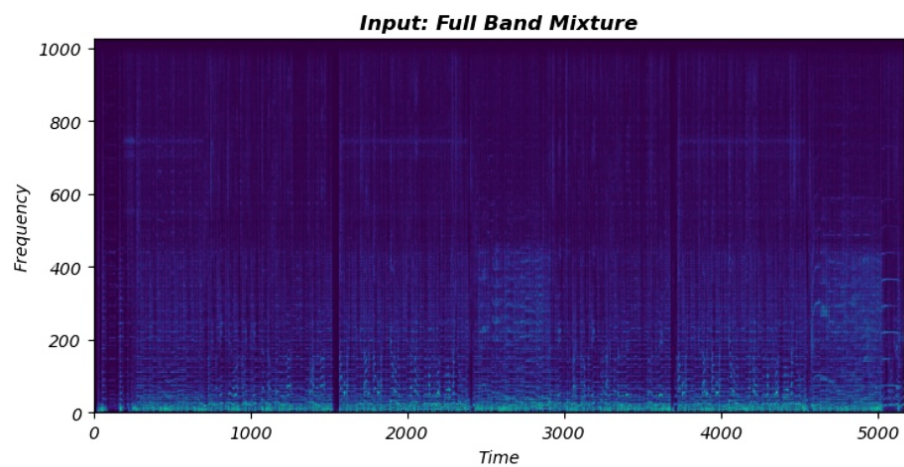


Figure 1: Spectrogram of 'Angels in Amplifiers' from the dataset

MUSDB18

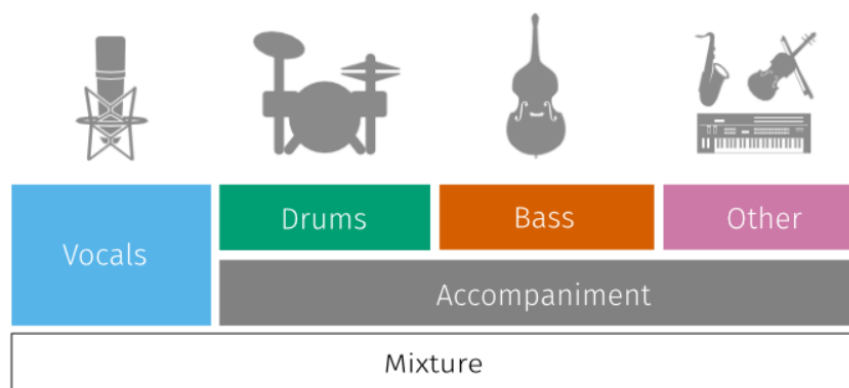


Figure 2: MUSDB18 tracks

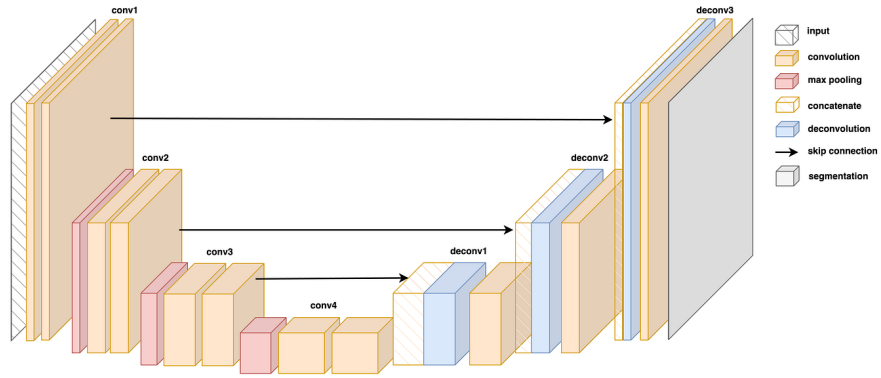


Figure 3: The U-Net architecture illustrated. The encoder and decoder are connected via skip connections.

- [2] A. Gupta, A. Rai, A. Bansal, and V. Arora, “Continual Learning for Singing Voice Separation with Human in the Loop Adaptation,” Indian Institute of Technology, Kanpur, India.